

Backstepping Swing-Up and Stabilization of the Acrobot

Project 3 — Advanced Control Methods

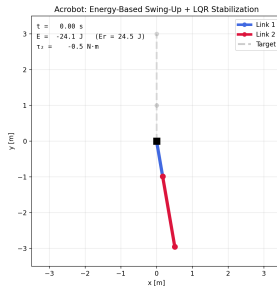
Mariya Lezina • Safina Gulyamova • Hajira Amjad • Ilya Mikhalchuk • Mikhail Sannikov

Skoltech, 2026

The Acrobot with Actuator Dynamics

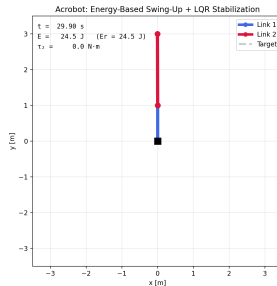
Same underactuated acrobot, new input path.

- Shoulder joint is **unactuated**
- Elbow torque τ_2 is an **actuator state**
- Command u is filtered:
$$T_a \dot{\tau}_2 = u - \tau_2$$
- $T_a = 0.005$ s, $|u| \leq 50$ N m
- State: $x = [q_1, q_2, \dot{q}_1, \dot{q}_2, \tau_2]^\top$
- Goal: swing up and stabilize using only u



Hanging (stable)

$$q_1 = -\frac{\pi}{2}$$



Upright (unstable)

$$q_1 = +\frac{\pi}{2}$$

Equations of Motion

$$M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + G(q) = \begin{bmatrix} 0 \\ \tau_2 \end{bmatrix}, \quad T_a \dot{\tau}_2 = u - \tau_2$$

Mechanical subsystem:

$$M = \begin{bmatrix} \alpha_1 + \alpha_2 + 2\alpha_3 \cos q_2 & \alpha_2 + \alpha_3 \cos q_2 \\ \alpha_2 + \alpha_3 \cos q_2 & \alpha_2 \end{bmatrix}$$

$$C\dot{q} = \alpha_3 \sin q_2 \begin{bmatrix} -2\dot{q}_1 \dot{q}_2 - \dot{q}_2^2 \\ \dot{q}_1^2 \end{bmatrix}$$

$$G = \begin{bmatrix} \beta_1 \cos q_1 + \beta_2 \cos(q_1 + q_2) \\ \beta_2 \cos(q_1 + q_2) \end{bmatrix}$$

Lumped parameters:

$$\alpha_1 = m_1 l_{c1}^2 + m_2 l_1^2 + I_1$$

$$\alpha_2 = m_2 l_{c2}^2 + I_2$$

$$\alpha_3 = m_2 l_1 l_{c2}$$

$$\beta_1 = (m_1 l_{c1} + m_2 l_1) g$$

$$\beta_2 = m_2 l_{c2} g$$

The actuator adds one state but does not change M , C , or G .

Backstepping Idea

Reuse Project 1 as the outer loop.

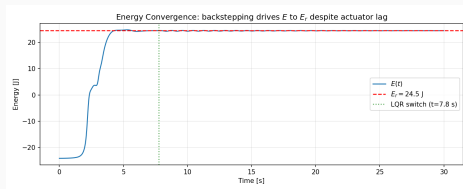
$$V_0 = \frac{1}{2}(E - E_r)^2 + \frac{1}{2} k_D \dot{q}_2^2 + \frac{1}{2} k_P q_2^2$$

If the elbow torque were direct, Project 1 would choose τ_2^* so that

$$\dot{V}_0 = -k_V \dot{q}_2^2 \leq 0.$$

In Project 3, τ_2^* becomes a **virtual control**: the desired actuator output, not the command.

Backstepping task: make $\tau_2(t)$ track $\tau_2^*(q, \dot{q})$ fast enough for the outer energy proof.



The same target energy $E_r = 24.5$ J is used, but actuator lag must be compensated before LQR can catch the upright pass.

Backstepping Control Law

Project 1's energy-shaping torque is reused as the virtual control $\tau_2^*(q, \dot{q})$; the command u makes the actuator output track it.

Inner tracking error:

$$z = \tau_2 - \tau_2^*$$

Command law:

$$u = \tau_2^* + T_a \dot{\tau}_2^* - k_z T_a z$$

Resulting error dynamics:

$$\dot{z} = -\left(\frac{1}{T_a} + k_z\right)z$$

Chain-rule feedforward:

$$\dot{\tau}_2^* = \sum_{k=1}^4 \frac{\partial \tau_2^*}{\partial x_{m,k}} \dot{x}_{m,k}$$

The mechanical acceleration uses the **actual** τ_2 , and the gradient is evaluated by central finite differences.

Gains:

$$k_D = 35.8, \quad k_P = 61.2, \quad k_V = 66.3, \quad k_z = 25$$

Inner time constant: ≈ 4.4 ms.

LQR Stabilization & Controller Switching

Augmented Lyapunov function:

$$V_3 = V_0 + \frac{1}{2}z^2$$

The inner loop makes z decay exponentially, so the outer energy controller recovers the Project 1 swing-up behavior.

Phase 2 — LQR at the upright:

Use the Project 1 mechanical LQR and add actuator damping:

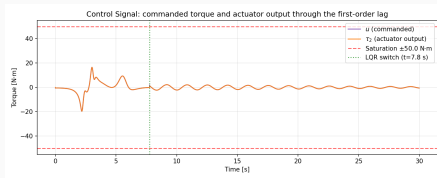
$$u = -K \begin{bmatrix} e_{q_1} & q_2 & \dot{q}_1 & \dot{q}_2 & \tau_2 \end{bmatrix}^T$$

$$K = \begin{bmatrix} -238.6 & -95.6 & -103.0 & -48.6 & 1.0 \end{bmatrix}$$

Switching strategy:

1. Backstepping swing-up runs.
2. Monitor mechanical error only:
 $\|e\|_w = |e_{q_1}| + |q_2| + 0.1|\dot{q}_1| + 0.1|\dot{q}_2|$
3. $\|e\|_w < 0.04 \Rightarrow$ switch to LQR.
4. One-way switch until $t = 30$ s.

Switch occurs at $t \approx 7.8$ s.



Implementation

Project structure:

<code>configs/params.py</code>	— params
<code>src/system.py</code>	— 5D dynamics
<code>src/controller.py</code>	— controllers
<code>src/simulation.py</code>	— switching ODE
<code>src/visualization.py</code>	— plots
<code>src/animation.py</code>	— GIF
<code>src/main.py</code>	— entry point

Reproduce:

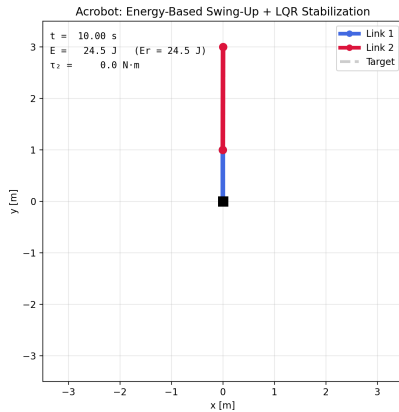
```
pip install -r requirements.txt
python -m src.main
python -m src.main --no-anim
```

Deps: numpy, scipy, matplotlib.

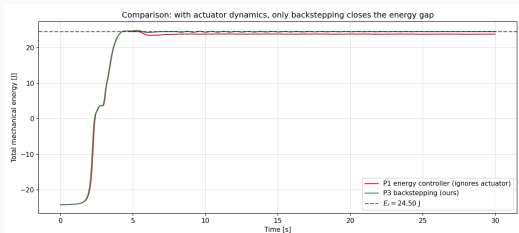
Algorithm:

1. Build 5D plant: $T_a \dot{\tau}_2 = u - \tau_2$
2. Verify inherited k_D solvability bound
3. Compute Project 1 virtual torque τ_2^*
4. Estimate $\dot{\tau}_2^*$ by chain rule
5. Command $u = \tau_2^* + T_a \dot{\tau}_2^* - k_z T_a z$
6. Switch to LQR when $\|e\|_w < 0.04$
7. Generate plots, comparisons, and animation

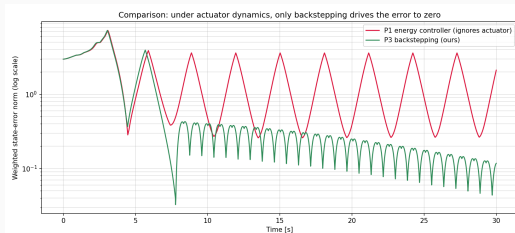
Animation



Results — Why Backstepping Is Needed

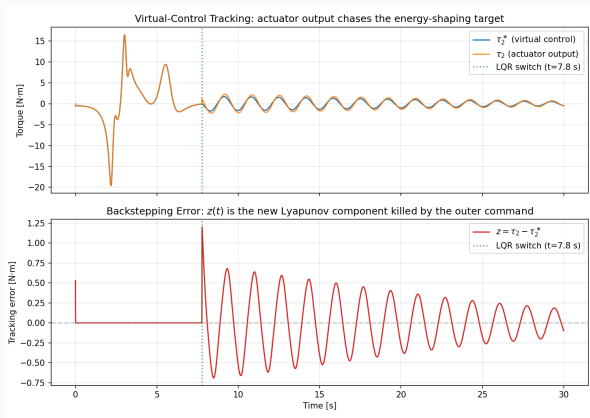


Both controllers pump energy, but the baseline does not settle at the upright energy after actuator lag is introduced.



Naive Project 1 control never crosses the 0.04 switch threshold; backstepping enters the LQR basin.

Results — Actuator Tracking

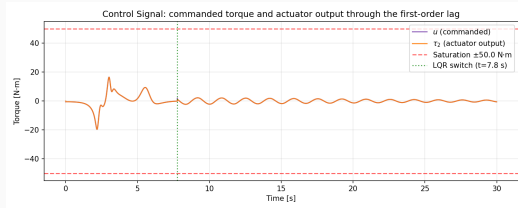


The actuator output τ_2 tracks the virtual control τ_2^* during swing-up; the visible step marks the handover from backstepping to LQR.

Results — Closed-Loop Response



Weighted mechanical error drops after the LQR switch and stays in the upright neighborhood.



Command u and actuator output τ_2 remain inside the ± 50 N m saturation limits.

Summary

What works:

- Backstepping compensates first-order actuator lag
- Torque-tracking error decays in ~ 4.4 ms
- Swing-up reaches the LQR basin at ~ 7.8 s
- Naive Project 1 baseline fails under actuator lag

Limitations:

- Final LQR residual is looser than Project 1
- Full 5D LQR gains saturate the command
- Larger T_a values need a stronger approach phase
- No friction or model uncertainty considered

Thank you. We are happy to take questions.